Modelli Lineari

Vediamo ora una delle possibili famiglie H utilizzabe: quella dei Modelli Lineari

Questa e' la piu' semplice ed utilizzata tra le famiglie H e consiste nel predire i target sfruttando una combinazione lineare di x

Ad esempio preso un'input d-dimensionale

$$x = (x_1, \dots, x_d)^T$$

Allora definiamo un predittore $h(x)$ come

D'ora in poi ci riferiremo al predittore con w

$$h(x; w, b) = x_1 w_1 + \dots + x_d w_d + b = w^T x + b \quad \text{con } w \in \mathbb{R}^d \text{ vettore dei pesi e } b \text{ costante}$$

Ci evita di avere predizioni nulle quando il vettore $x = 0$

Oss

Definendo cosi il modello abbiamo una linearita' sotto due punti di vista differenti:

i) lineare secondo le feature x

ii) lineare secondo i parametri w e b

Rappresentazione delle Feature

All'interno dei modelli lineari una problematica ricorrente e' la rappresentazione delle feature infatti:

i) Una pessima rappresentazione aumenta il carico di lavoro e quindi porta a causare piu' errori

ii) Una buona rappresentazione rende piu' visibile il pattern da stimare e semplifica il processo di Learning

La branca che studia la rappresentazione delle feature e' chiamata Feature engineering e studia come mappare l'input $x \in \mathbb{R}^d$ in $\mathbb{R}^m$ sfruttando un'insieme di m funzioni base $\phi_1, \dots, \phi_m$ in modo da avere

Prende tutto il vettore

$$h(x; w, b) = \sum_{i=1}^{m} w_i \phi_i(x) + b = w^T \phi(x) + b \quad \text{con } \phi(x) = (\phi_1(x), \dots, \phi_m(x))^T$$

In questo modo relazioni non lineari in $\mathbb{R}^d$ lo diventano in $\mathbb{R}^m$

Manteniamo la linearita' secondo w

Design Matrix

Ricordando che la matrice

$$X = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ \vdots & & & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{pmatrix}$$

Rappresenta gli n vettori in input nel $\mathcal{X}$ dove all'i-esimo riga x_i^T e' un vettore di d features

Applicando $\underline{\phi} = (\phi_1, \dots, \phi_M)$ ad ogni vettore $x_i \in X$ otteniamo la Design Matrix $\tilde{\Phi}$

$$\tilde{\Phi} = \begin{pmatrix} \phi_1(x_1) & \phi_2(x_1) & \cdots & \phi_M(x_1) \\ \vdots & & & \\ \phi_1(x_n) & \phi(x_n) & \cdots & \phi_M(x_n) \end{pmatrix}$$

D'ora in poi useremo X per indicare la Design Matrix

La scelta di M e' molto importante in quanto un M basso solitamente porta ad underfitting mentre un M alto porta ad overfitting

Quadratic Loss

detta anche Quadratic Loss

All'interno dei modelli lineari la Loss piu' utilizzata e' quella degli errori quadrati

$$L(y,t) = (y-t)^2$$

La quale presenta diverse proprieta':

i) Gli errori piu' grandi contribuiscono di piu' alla Loss rispetto a quelli piu' piccoli

ii) La Quadratic Loss e' convessa e differenziabile $\forall y \in Y$ → Presenta un'unico minimo globale

Definendo cosi' la Loss abbiamo il rispettivo rischio empirico

$$\hat{R}_{\hat{\mathcal{L}}}(h) = \frac{1}{|\hat{\mathcal{L}}|} \sum_{(x,t) \in \hat{\mathcal{L}}} (h(x), t)^2$$

Bias-Variance Decomposition

Tramite la bias-Variance decomposition riusciamo a capire quanto errore sia ineribabile e quanto sia riducibile

Ponendo $\mu(x) = \mathop{\mathbb{E}}_{t \sim p_t}[t]$ e $\sigma^2(x) = \mathop{Var}_{t \sim p_t}(t)$ Abbiamo per il rischio atteso $R(x;h)$

$$R(x,h) = \mathop{\mathbb{E}}_{t \sim p_t}\left[(h(x)-t)^2\right] = \mathop{\mathbb{E}}_{t \sim p_t}\left[(h(x)+\mu(x)-\mu(x)-t)^2\right] = (h(x)-\mu(x))^2 + 2(h(x)-t)\mathop{\mathbb{E}}_{t \sim p_t}[\mu(x)-t] + \mathop{\mathbb{E}}_{t \sim p_t}\left[(\mu(x)-t)^2\right] =$$

(o visto che $\mathbb{E}[t] = \mu(x)$)

$$= (h(x)-\mu(x))^2 + \mathop{\mathbb{E}}_{t \sim p_t}\left[(\mu(x)-t)^2\right]$$

bias

Varianza $= \sigma^2(x)$

Ponendo il predittore Bayesiano $h^B(x)=\mu$ vediamo come il fattore riducibile sia $(h(x)-\mu)^2$ mentre σ^2 rimane irriducibile

Dobbiamo questa però noi il rischio atteso lo studiamo secondo il predittore $h_{\tilde{\mathcal{L}}}$ trovato scegliendo $\tilde{\mathcal{L}}$ random

Ponendo $\bar{h}(x)=\mathop{\mathbb{E}}_{\tilde{\mathcal{L}}}[h_{\tilde{\mathcal{L}}}(x)]$ ~ Diverso da $\bar{h}(x)$ delle dispense prima

Abbiamo una Bias-Variance decomposition della forma

$$\mathop{\mathbb{E}}_{\tilde{\mathcal{L}}}\left[R(x, h_{\tilde{\mathcal{L}}})\right]=\sigma^2(x)+\left(\bar{h}(x)-\mu(x)\right)^2+\mathop{Var}_{\tilde{\mathcal{L}}}\left[h_{\tilde{\mathcal{L}}}(x)\right]$$

Dove

i) $\sigma^2(x)$ essendo irriducibile indica il Noise del sistema

ii) $(\bar{h}(x)-\mu(x))$ indica il Bias e ci dice quanto in media il nostro predittore è lontano dal migliore

iii) $\mathop{Var}_{\tilde{\mathcal{L}}}[h_{\tilde{\mathcal{L}}}(x)]$ indica quanto il predittore sia sensibile da $\tilde{\mathcal{L}}$

Least Squares

$$\hat{R}_{\tilde{\mathcal{L}}}^{(n)}\frac{1}{\tilde{\mathcal{L}}}\sum\ell(h,x) \rightsquigarrow \frac{1}{|\tilde{\mathcal{L}}|}\sum_{i=1}^{n}\left(h(x_i)-\vec{w}\right)^2$$ → Least squares

Sotto la Loss quadratica il rischio empirico è proporzionale alla somma dei residui quadrati

$$\bar{C}(w,b)=\frac{1}{n}\sum_{i=1}^{n}r_i^2 \quad \text{con} \quad r_i=\sum_{j=1}^{d}w_j x_{ij}+b-t_i$$ → $\bar{C}$ convessa $\Longrightarrow$ Un'unico minimo globale

Oss

$$\hat{R}_{\tilde{\mathcal{L}}}(h)=\frac{1}{n}\sum_{i=1}^{n}(h(x_i)-t_i)^2=\frac{1}{n}\sum_{i=1}^{n}\left(\sum_{j=1}^{d}w_j x_{ij}+b-t_i\right)^2=\frac{1}{n}\sum_{i=1}^{n}r_i^2=\bar{C}(w,b)$$

Data la natura quadratica di $\bar{C}$ il suo gradiente risulta essere lineare nei parametri e, ponendo il tutto uguale a 0, ci porta ad avere un sistema con $d+1$ equazioni lineari

Invece di risolvere il sistema in forma lineare è possibile scrivere il sistema in forma matriciale:

i) Aggiungere il bias al vettore w

$$\bar{w}=(b, w_1, \dots, w_d)^T$$

ii) Aggiungere una colonna di 1 alla Design Matrix

$$\bar{X}=\begin{pmatrix} 1 & x_{11} & \cdots & x_{1d} \\ \vdots & \vdots & & \vdots \\ 1 & x_{n1} & \cdots & x_{nd} \end{pmatrix}$$

In questo modo il vettore delle predizioni prende la forma di $\bar{X}\bar{w}$ con rispettivo errore $C(\bar{w}) = \|\bar{X}\bar{w} - \bar{c}\|^2$

Andando ad eseguire il gradiente di $C(\bar{w})$ e lo poniamo uguale a 0 otteniamo

$$\bar{X}^T(\bar{X}\bar{w} - \bar{c}) = 0 \iff \bar{X}^T\bar{X}\bar{w} = \bar{X}^T\bar{c}$$

$$\iff \bar{w} = (\bar{X}^T\bar{X})^{-1}\bar{X}^T\bar{c}$$

In poche parole attraverso questa scrittura otteniamo una formula chiusa per il calcolo di $\bar{w}$ senza eseguire la discesa del gradiente

Ovviamente questa formula è applicabile solo per valori bassi di d visto che eseguire un calcolo del genere ha complessità $O(d^3)$

Regolarizzazione

La riduzione dell'overfitting è una questione delicata, fino ad ora lo abbiamo gestito

i) Aumentando la dimensione di $\tilde{c}$ ⇝ A volte però non si dispone sempre di tutti i dati desiderati

ii) Restringendo la famiglia di predittori H ⇝ Diminuendo il numero M di funzioni base

Un altro modo passa attraverso la Regolazione, di cui ne esistono 2 tipi:

1) Hard regolarization:

   Consiste nel settare a 0 di default delle features

2) Soft regolarization:

   Aggiunge una penalità proporzionale ai valori dei parametri

Soft regolarization

Questa tecnica va a modificare il calcolo dietro l'errore

$$C(\bar{w}) = C_D(\bar{w}) + \lambda C_w(\bar{w})$$

Dove

i) $C_D(\bar{w})$ è il nostro rischio empirico

ii) $C_w(\bar{w})$ è il regolarization term

iii) λ è il regolarization coefficient

Oss

Tornando a Bias e Varianza

i) λ alto porta un bias molto basso ma varianza alta → overfitting

a) λ basso porta un bias molto alto e una varianza bassa → underfitting

La formula generale per il regolarization term è la seguente:

$$C_w(\vec{w}) = \frac{\lambda}{2} \sum_{i=1}^{d} |w_i|^q \quad \text{con } q \in \{1, 2\}$$

Dove q è un parametro che gestisce quanto vogliamo penalizzare il modello, in fatti

i) Per $q=1$ abbiamo la Lasso:

Riforma soluzioni sparse con molti pesi posti a 0

a) Per $q=2$ abbiamo la Ridge regression:

Abbassa i pesi verso lo 0 senza annullarli del tutto